UCS301 Data Structures

Lab Assignment 4: Queues (Week 4)

Name: Nandini Sharma

Roll No: 2026010076

Group: MCA3

Question 1: Menu Driven Program for Simple Queue

Operations: enqueue(), dequeue(), isEmpty(), isFull(), display(), peek()

Program:

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

int isEmpty() { return front == -1; }
int isFull()  { return rear == MAX - 1; }

void enqueue(int x) {
    if (isFull()) { printf("Queue is FULL\n"); return; }
    if (front == -1) front = 0;
    rear++;
    queue[rear] = x;
    printf("%d inserted\n", x);
}

void dequeue() {
    if (isEmpty()) { printf("Queue is EMPTY\n"); return; }
    printf("%d deleted\n", queue[front]);
    if (front == rear) front = rear = -1;   /* last element removed */
    else front++;
}

void peek() {
    if (isEmpty()) printf("Queue is EMPTY\n");
    else printf("Front element = %d\n", queue[front]);
}

void display() {
    int i;
    if (isEmpty()) { printf("Queue is EMPTY\n"); return; }
    printf("Queue: ");
    for (i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int ch, x;
    do {
        printf("\n1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek
7.Exit\n");
        printf("Enter choice: ");
```

```
        scanf("%d", &ch);
        if (ch == 1) { printf("Enter value: "); scanf("%d", &x); enqueue(x); }
        else if (ch == 2) dequeue();
        else if (ch == 3) { if (isEmpty()) printf("Yes, empty\n"); else printf("No,
not empty\n"); }
        else if (ch == 4) { if (isFull()) printf("Yes, full\n"); else printf("No,
not full\n"); }
        else if (ch == 5) display();
        else if (ch == 6) peek();
    } while (ch != 7);
    return 0;
}
```

Output:

```
1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 20
20 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 5
Queue: 10 20 30

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 6
Front element = 10

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 2
10 deleted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 5
Queue: 20 30

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 3
No, not empty

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 4
No, not full

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 40
40 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 50
```

```
50 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 60
Queue is FULL

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 70
Queue is FULL

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 4
Yes, full

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 7
```

Question 2: Menu Driven Program for Circular Queue

Operations: enqueue(), dequeue(), isEmpty(), isFull(), display(), peek()

Program:

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

int isEmpty() { return front == -1; }
int isFull()  { return (rear + 1) % MAX == front; }

void enqueue(int x) {
    if (isFull()) { printf("Queue is FULL\n"); return; }
    if (isEmpty()) front = rear = 0;
    else rear = (rear + 1) % MAX;
    queue[rear] = x;
    printf("%d inserted\n", x);
}

void dequeue() {
    if (isEmpty()) { printf("Queue is EMPTY\n"); return; }
    printf("%d deleted\n", queue[front]);
    if (front == rear) front = rear = -1;
    else front = (front + 1) % MAX;
}

void peek() {
    if (isEmpty()) printf("Queue is EMPTY\n");
    else printf("Front element = %d\n", queue[front]);
}

void display() {
    int i;
    if (isEmpty()) { printf("Queue is EMPTY\n"); return; }
    printf("Queue: ");
    i = front;
```

```
    while (i != rear) {
        printf("%d ", queue[i]);
        i = (i + 1) % MAX;
    }
    printf("%d\n", queue[rear]);
}

int main() {
    int ch, x;
    do {
        printf("\n1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek
7.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);
        if (ch == 1) { printf("Enter value: "); scanf("%d", &x); enqueue(x); }
        else if (ch == 2) dequeue();
        else if (ch == 3) { if (isEmpty()) printf("Yes, empty\n"); else printf("No,
not empty\n"); }
        else if (ch == 4) { if (isFull()) printf("Yes, full\n"); else printf("No,
not full\n"); }
        else if (ch == 5) display();
        else if (ch == 6) peek();
    } while (ch != 7);
    return 0;
}
```

Output:

```
1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 20
20 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 40
40 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 50
50 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 60
Queue is FULL

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 4
Yes, full
```

```
1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 2
10 deleted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 2
20 deleted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 1
Enter value: 60
60 inserted

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 5
Queue: 30 40 50 60

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 6
Front element = 30

1.Enqueue 2.Dequeue 3.isEmpty 4.isFull 5.Display 6.Peek 7.Exit
Enter choice: 7
```

Question 3: Interleave the First Half of the Queue with the Second Half

Sample I/P: 4 7 11 20 5 9 Sample O/P: 4 20 7 5 11 9

Program:

```
/* Interleave first half of queue with second half */
#include <stdio.h>

int q[50], front = 0, rear = -1;      /* main queue   */
int h[50], hf = 0, hr = -1;           /* first-half queue */

int main() {
    int n, i, x;
    printf("Enter n (even): ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) { scanf("%d", &x); rear++; q[rear] = x; }

    /* move first half to queue h */
    for (i = 0; i < n / 2; i++) { hr++; h[hr] = q[front]; front++; }

    /* take one from h, then one from q, and put back in q */
    for (i = 0; i < n / 2; i++) {
        rear++; q[rear] = h[hf]; hf++;
        rear++; q[rear] = q[front]; front++;
    }

    printf("Output: ");
    while (front <= rear) { printf("%d ", q[front]); front++; }
    printf("\n");
    return 0;
}
```

Output:

```
Enter n (even): 6
Enter 6 elements: 4 7 11 20 5 9
Output: 4 20 7 5 11 9
```

Question 4: First Non-Repeating Character in a String using Queue

Sample I/P: a a b c Sample O/P: a -1 b b

Program:

```
/* First non-repeating character using a queue */
#include <stdio.h>

int main() {
    char s[50], queue[50];
    int count[256] = {0};
    int front = 0, rear = -1, i;

    printf("Enter string: ");
    scanf("%s", s);

    printf("Output: ");
    for (i = 0; s[i] != '\0'; i++) {
        count[(int)s[i]]++;                 /* count this character */
        rear++; queue[rear] = s[i];    /* enqueue it           */

        /* remove front characters that are repeated */
        while (front <= rear && count[(int)queue[front]] > 1)
            front++;

        if (front > rear) printf("-1 ");
        else printf("%c ", queue[front]);
    }
    printf("\n");
    return 0;
}
```

Output:

```
Enter string: aabc
Output: a -1 b b
```

Question 5 (a): Implement a Stack using Two Queues

Operations: push(), pop(), top(), display()

Program:

```
/* Stack using TWO queues (push is costly) */
#include <stdio.h>
#define MAX 50
```

```
int q1[MAX], f1 = 0, r1 = -1;     /* main queue, front = top of stack */
int q2[MAX], f2 = 0, r2 = -1;     /* helper queue */

void push(int x) {
    r2++; q2[r2] = x;                          /* 1. new element into q2 */
    while (f1 <= r1) { r2++; q2[r2] = q1[f1]; f1++; }   /* 2. move all of q1 behind
it */
    f1 = 0; r1 = -1;
    while (f2 <= r2) { r1++; q1[r1] = q2[f2]; f2++; }   /* 3. move everything back
to q1 */
    f2 = 0; r2 = -1;
    printf("%d pushed\n", x);
}

void pop() {
    if (f1 > r1) { printf("Stack is EMPTY\n"); return; }
    printf("%d popped\n", q1[f1]);
    f1++;
}

void top() {
    if (f1 > r1) printf("Stack is EMPTY\n");
    else printf("Top = %d\n", q1[f1]);
}

void display() {
    int i;
    if (f1 > r1) { printf("Stack is EMPTY\n"); return; }
    printf("Stack (top to bottom): ");
    for (i = f1; i <= r1; i++) printf("%d ", q1[i]);
    printf("\n");
}

int main() {
    int ch, x;
    do {
        printf("\n1.Push 2.Pop 3.Top 4.Display 5.Exit\nEnter choice: ");
        scanf("%d", &ch);
        if (ch == 1) { printf("Enter value: "); scanf("%d", &x); push(x); }
        else if (ch == 2) pop();
        else if (ch == 3) top();
        else if (ch == 4) display();
    } while (ch != 5);
    return 0;
}
```

Output:

```
1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 10
10 pushed

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 20
20 pushed

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 30
30 pushed
```

```
1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 30 20 10

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 3
Top = 30

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
30 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 20 10

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
20 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
10 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
Stack is EMPTY

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 5
```

Question 5 (b): Implement a Stack using One Queue

Operations: push(), pop(), top(), display()

Program:

```
/* Stack using ONE queue (push is costly) */
#include <stdio.h>
#define MAX 50

int q[MAX];
int front = 0, rear = -1, count = 0;

void enqueue(int x) { rear = (rear + 1) % MAX; q[rear] = x; count++; }
int  dequeue()      { int x = q[front]; front = (front + 1) % MAX; count--; return
x; }

void push(int x) {
    int i, n = count;
    enqueue(x);                          /* add new element at rear */
    for (i = 0; i < n; i++)              /* rotate old elements behind it */
        enqueue(dequeue());
    printf("%d pushed\n", x);
}

void pop() {
    if (count == 0) { printf("Stack is EMPTY\n"); return; }
```

```
    printf("%d popped\n", dequeue());
}

void top() {
    if (count == 0) printf("Stack is EMPTY\n");
    else printf("Top = %d\n", q[front]);
}

void display() {
    int i, k = front;
    if (count == 0) { printf("Stack is EMPTY\n"); return; }
    printf("Stack (top to bottom): ");
    for (i = 0; i < count; i++) { printf("%d ", q[k]); k = (k + 1) % MAX; }
    printf("\n");
}

int main() {
    int ch, x;
    do {
        printf("\n1.Push 2.Pop 3.Top 4.Display 5.Exit\nEnter choice: ");
        scanf("%d", &ch);
        if (ch == 1) { printf("Enter value: "); scanf("%d", &x); push(x); }
        else if (ch == 2) pop();
        else if (ch == 3) top();
        else if (ch == 4) display();
    } while (ch != 5);
    return 0;
}
```

Output:

```
1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 10
10 pushed

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 20
20 pushed

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 1
Enter value: 30
30 pushed

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 30 20 10

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 3
Top = 30

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
30 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 20 10
```

```
1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
20 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
10 popped

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 2
Stack is EMPTY

1.Push 2.Pop 3.Top 4.Display 5.Exit
Enter choice: 5
```